

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL & DEBUGGING

Oleh:

Abdurrahman Gilang Harjuna

NIM. 2310817110004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel & Debugging List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Abdurrahman Gilang Harjuna
NIM : 2310817110004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code	6
B. Output Program.....	8
C. Pembahasan	10
D. Tautan Git	11

DAFTAR GAMBAR

Gambar 1. Contoh Penggunaan Debugger	6
Gambar 2. Screenshot hasil soal 1.....	8
Gambar 3. Screenshot hasil soal 1.....	9
Gambar 4. Screenshot hasil soal 1.....	9
Gambar 5. Screenshot hasil soal 1.....	10

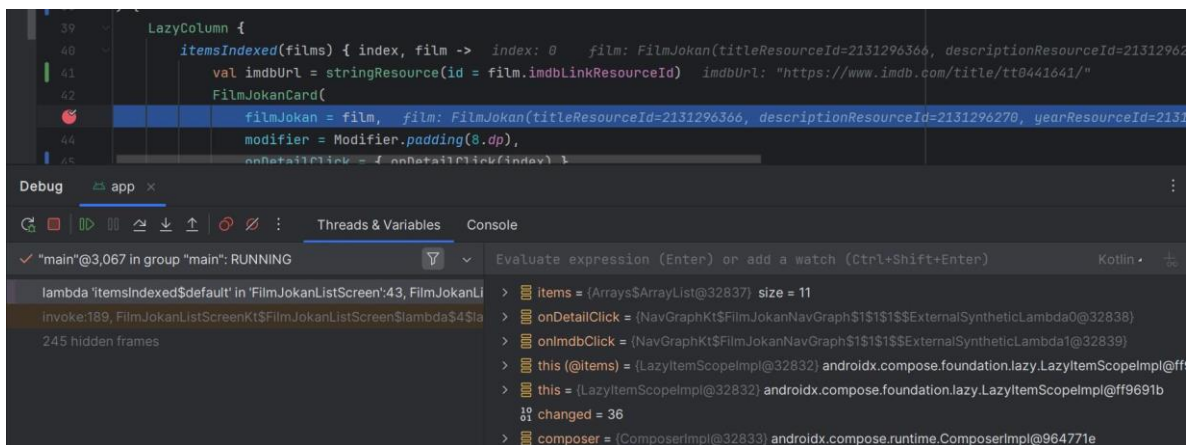
DAFTAR TABEL

Table 1. Source Code Jawaban Soal 1.....	7
Table 2. Source Code Jawaban Soal 1.....	7
<i>Table 3. Source Code Jawaban Soal 1</i>	<i>8</i>

SOAL 1

1. Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:
 - a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data
 - b. tidak boleh disimpan langsung di dalam Fragment atau Activity.
 - c. Gunakan ViewModelFactory dalam pembuatan ViewModel
 - d. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out
2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 1. Contoh Penggunaan Debugger

A. Source Code

1. Alat Gym.kt

```
1 package com.example.zennfit
```

3	<code>data class AlatGym(</code>
4	<code> val id: Int,</code>
5	<code> val nama: String,</code>
6	<code> val deskripsiResId: Int,</code>
7	<code> val imageResId: Int,</code>
8	<code> val gifResId: Int,</code>
	<code>)</code>

Table 1. Source Code Jawaban Soal 1

2. AlatGymViewModel

1	<code>package com.example.zennfit</code>
2	
3	<code>import androidx.lifecycle.ViewModel</code>
4	<code>import kotlinx.coroutines.flow.MutableStateFlow</code>
5	<code>import kotlinx.coroutines.flow.StateFlow</code>
6	<code>import kotlinx.coroutines.flow.asStateFlow</code>
7	<code>class AlatGymViewModel : ViewModel() {</code>
8	<code> private val _alatList =</code>
9	<code>MutableStateFlow<List<AlatGym>>(emptyList())</code>
10	<code> val alatList: StateFlow<List<AlatGym>> = _alatList.asStateFlow()</code>
11	
	<code> init {</code>
	<code> loadAlatGym()</code>
	<code> }</code>
	<code> private fun loadAlatGym() {</code>
	<code> val data = listOf(</code>
	<code> AlatGym(1, "BenchPress", R.string.desc_benchpress,</code>
	<code>R.drawable.benchpress, R.drawable.incline_benchpress),</code>
	<code> AlatGym(2, "Row Cable", R.string.desc_row,</code>
	<code>R.drawable.cablerow, R.drawable.rowbro),</code>
	<code> AlatGym(3, "Shoulderpress Machine",</code>
	<code>R.string.desc_shoulder, R.drawable.shoulderpress,</code>
	<code>R.drawable.shoulderpress_machine),</code>
	<code> AlatGym(4, "Legpress Machine", R.string.desc_legpress,</code>
	<code>R.drawable.legpress, R.drawable.legpressmachine),</code>
	<code> AlatGym(5, "Fly Machine", R.string.desc_fly,</code>
	<code>R.drawable.fly, R.drawable.fly_machine)</code>
	<code>)</code>
	<code> _alatList.value = data</code>
	<code> }</code>
	<code>}</code>

Table 2. Source Code Jawaban Soal 1

1. AlatGymViewModel

1	<code>package com.example.zennfit</code>
2	
3	<code>import androidx.lifecycle.ViewModel</code>
4	<code>import androidx.lifecycle.ViewModelProvider</code>

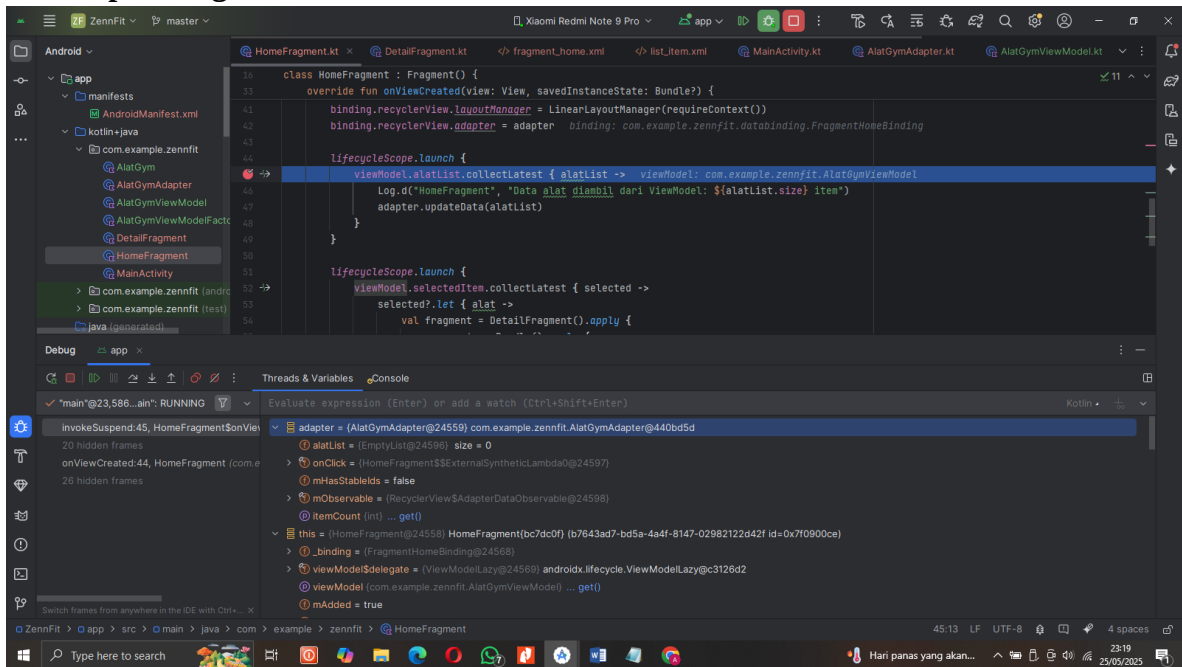
```

5 class AlatGymViewModelFactory : ViewModelProvider.Factory {
6     override fun <T : ViewModel> create(modelClass: Class<T>): T {
7         if
8         (modelClass.isAssignableFrom(AlatGymViewModel::class.java)) {
9             return AlatGymViewModel() as T
10        }
11        throw IllegalArgumentException("Unknown ViewModel class")
12    }
13 }

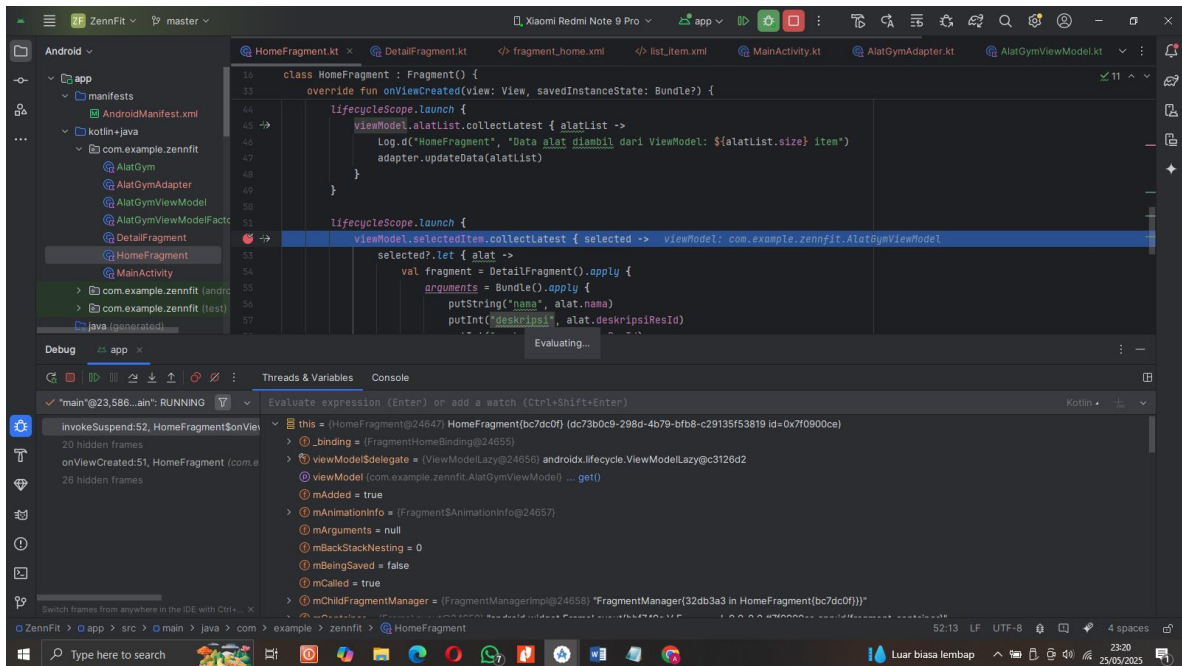
```

Table 3. Source Code Jawaban Soal 1

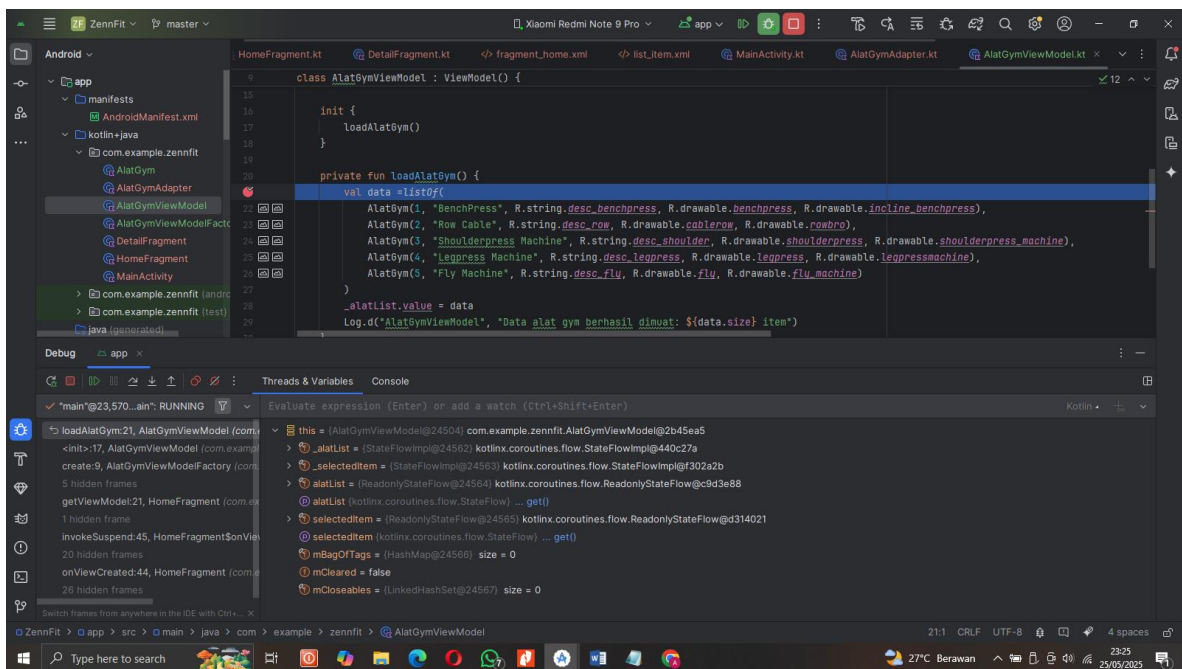
B. Output Program



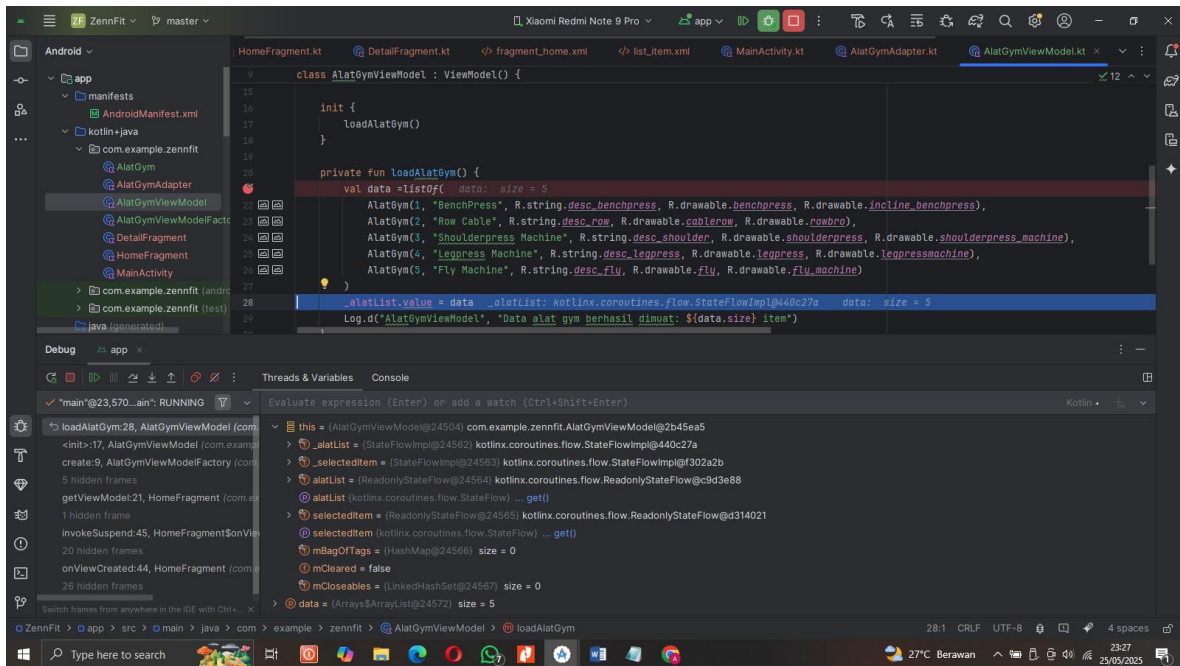
Gambar 2. Screenshot hasil soal 1



Gambar 3. Screenshot hasil soal 1



Gambar 4. Screenshot hasil soal 1



Gambar 5. Screenshot hasil soal 1

C. Pembahasan

Debugger adalah alat di Android Studio yang membantu developer melihat jalannya aplikasi secara rinci saat runtime, menghentikan eksekusi di titik tertentu (breakpoint), dan memeriksa nilai variabel serta alur program.

Cara menggunakan Debugger:

1. Letakkan breakpoint dengan klik kiri di margin kiri editor.
2. Jalankan aplikasi dalam mode Debug (klik ikon bug atau Shift+F9).
3. Ketika eksekusi sampai breakpoint, aplikasi berhenti dan developer dapat memeriksa variabel, thread, dan call stack.
4. Gunakan fitur stepping untuk menelusuri kode.
- 5.

Fitur stepping:

1. **Step Into:** Masuk ke dalam fungsi/method yang dipanggil, melihat implementasinya baris demi baris.
2. **Step Over:** Lewati fungsi/method yang dipanggil, langsung lanjut ke baris berikutnya di fungsi saat ini.
3. **Step Out:** Keluar dari fungsi/method saat ini dan kembali ke fungsi pemanggil.

Application Class

Application class adalah kelas dasar yang mewakili keseluruhan aplikasi Android. Kelas ini dijalankan pertama kali sebelum komponen lain (Activity, Service, dll). Fungsinya untuk menginisialisasi resource atau konfigurasi yang bersifat global, misalnya:

- Inisialisasi library seperti database, logging, dependency injection (Dagger/Hilt).
- Menyimpan state atau data yang ingin diakses secara global oleh seluruh komponen aplikasi.
- Mengelola lifecycle aplikasi secara keseluruhan.

Biasanya developer membuat subclass Application untuk melakukan setup awal dan mendaftarkannya di AndroidManifest.xml. Dengan Application class, kode konfigurasi tidak perlu diulang-ulang di setiap Activity atau Fragment.

1. AlatGym.kt

Isi dalamnya adalah deklarasi dari sebuah data bernama AlatGym, bisa dilihat ada val id: Int, val nama: String, val deskripsiResId: Int, val imageResId: Int, val gifResId: Int,

2. AlatGymViewModel

ViewModel ini tempat menyimpan data list alat gym yang sudah dimodifikasi yang mana sebelumnya disimpan di dalam Home fragment. Jadi deklarasi class AlatGymViewModel yang mana ini adalah subclass dari ViewModel. Lalu ada property privat _alatlist dan property public alatList yang hanya bisa dibaca dari luar. Lalu menaggil blok LoadAlatGym untuk Mengisi data alat gymnya . Fungsi ini membuat listalat gym secara manual lalu menyimpannya kedalam _alatList.

3. AlatGymViewModelFactory

Kode ini masih default karena tidak ada parameter pada viewModel.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Specter735/Pemrograman-Mobile.git>